

AI-gestuurd Dependency Dashboard voor Automatische Analyse en Prioritering van Software-updates.

Optionele ondertitel.

Jelle Vandriessche.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. T. Aelbrecht

Co-promotor: Dhr. B. Pattyn

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Samenvatting

Deze bachelorproef ontwikkelt een aparte webapplicatie (dashboard) met een from-scratch ontworpen AI-agent om dependency-updates automatisch te detecteren, analyseren en prioriteren voor het kernteam dat platform- en dependencybeheer opneemt binnen een bedrijf. Vanuit de vastgestelde problematiek — veel manueel werk bij het interpreteren van changelogs, ophopende technische schuld en beperkte context bij updates — controleert de applicatie GitHub-repositories op gebruikte dependencies, haalt versie-informatie op uit publieke registries (zoals npm) en laat de AI-agent changelogs en release notes samenvatten met indicatie van criticiteit (security, bugfix, feature) en risico-impact.

Via een systematische aanpak met requirements-analyse, architectuurontwerp, proof-of-concept implementatie en gerichte vergelijking van AI-aanpakken wordt onderzocht hoe deze oplossing de manuele beoordelingslast kan verminderen en het overzicht op technische schuld kan verbeteren. De evaluatie met het kernteam focust op bruikbaarheid van het dashboard, duidelijkheid van AI-samenvattingen en ondersteuning bij prioritering. Verwacht wordt dat de aanpak een efficiënter, transparanter dependencybeheerproces oplevert dat herhaalbaar is en kan worden opgeschaald naar andere teams binnen de organisatie.

Inhoudsopgave

Lijst van figuren	vii
Lijst van tabellen	viii
Lijst van codefragmenten	ix
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	2
1.3 Onderzoeksdoelstelling	3
1.4 Opzet van deze bachelorproef	4
2 Stand van zaken	5
2.0.1 Dependency management en kwetsbare dependencies	5
2.0.2 Technische schuld en onderhoudslast	5
2.0.3 Dependencybots en hun beperkingen	6
2.0.4 AI, agents en workflow automatisering	6
2.0.5 Onderzoeksniche: AI-ondersteund dependencybeheer	8
3 Methodologie	9
3.0.1 Fase 1 - Verdiepende literatuurstudie en probleemverkenning	9
3.0.2 Fase 2 - Requirements-analyse en architectuurontwerp	9
3.0.3 Fase 3 - Implementatie van het proof-of-concept	10
3.0.4 Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak	11
3.0.5 Fase 5 - Evaluatie en rapportering	12
4 Conclusie	14
A Onderzoeksvoorstel	16
A.1 Inleiding	16
A.1.1 Probleemstelling	16
A.1.2 Onderzoeksvraag	17
A.1.3 Onderzoeksdoelstelling	18
A.2 Literatuurstudie	19
A.2.1 Dependency management en kwetsbare dependencies	19
A.2.2 Technische schuld en onderhoudslast	20
A.2.3 Dependencybots en hun beperkingen	20

A.2.4	AI, agents en workflow automatisering	20
A.2.5	Onderzoeksniche: AI-ondersteund dependencybeheer	21
A.3	Methodologie	22
A.3.1	Fase 1 - Verdiepende literatuurstudie en probleemverkenning. .	22
A.3.2	Fase 2 - Requirements-analyse en architectuurontwerp	22
A.3.3	Fase 3 - Implementatie van het proof-of-concept	23
A.3.4	Fase 4 - Vergelijkende onderbouwing van architectuur en AI- aanpak.	23
A.3.5	Fase 5 - Evaluatie en rapportering.	24
A.4	Verwacht resultaat en conclusie	25

Bibliografie**27**

Lijst van figuren

3.1 Fasering en tijdsplanning van het onderzoek	13
A.1 Fasering en tijdsplanning van het onderzoek	26

Lijst van tabellen

Lijst van codefragmenten

1

Inleiding

Softwareprojecten binnen bedrijven en organisaties maken gebruik van talrijke externe bibliotheken, frameworks en tools, de zogenaamde *dependencies*. Deze dependencies worden voortdurend bijgewerkt om nieuwe functionaliteiten, verbeterde prestaties en vooral beveiligingspatches aan te bieden.(Pashchenko et al., 2018) In de praktijk blijkt het voor ontwikkelteams echter moeilijk om alle updates systematisch op te volgen en grondig te evalueren, zeker wanneer een bedrijf meerdere projecten en repositories beheert.

1.1. Probleemstelling

Het manueel opvolgen van dependency-updates vormt een groot probleem in softwareontwikkeling. Ontwikkelaars moeten regelmatig changelogs doorzoeken, compatibiliteit controleren en inschatten welke updates veilig kunnen worden doorgevoerd, wat in de praktijk veel tijd vraagt en vaak wordt uitgesteld.(Behutiye et al., 2024; Pashchenko et al., 2018) Onderzoek toont aan dat veel softwareprojecten hun dependencies niet systematisch bijhouden, waardoor een aanzienlijk deel verouderd raakt. Pashchenko et al. (Pashchenko et al., 2018) stellen bijvoorbeeld dat een belangrijk percentage kwetsbare dependencies relatief eenvoudig opgelost kan worden door een update, maar dat dit in de praktijk niet gebeurt wegens tijdsdruk en gebrek aan overzicht. Wanneer het updateproces niet systematisch gebeurt, stapelen onderhoudstaken zich op, waardoor het steeds moeilijker en tijdrovender wordt om de software up-to-date te houden. Dit fenomeen staat bekend als *technische schuld* (technical debt): de achterstand die ontstaat doordat noodzakelijke verbeteringen of updates te lang worden uitgesteld, wat leidt tot hogere onderhoudskosten en complexiteit op lange termijn.(Behutiye et al., 2024; Ruiz et al., 2024) Empirisch onderzoek bevestigt dat technische schuld binnen bedrijven reële negatieve gevolgen heeft voor productiviteit en softwarekwaliteit.(Besker,

2020) Om deze last te verlichten, zetten veel organisaties al geautomatiseerde dependencybots in, zoals Dependabot¹ of Renovate.² Deze tools maken wel automatisch pull requests aan zodra er nieuwe versies beschikbaar zijn, (Erlenhov et al., 2022) maar geven doorgaans weinig context over de inhoud en impact van de wijziging. Ontwikkelaars verliezen daardoor nog steeds tijd aan het interpreteren van changelogs, het inschatten van risico's en het beslissen of een update al dan niet moet worden doorgevoerd. (Erlenhov et al., 2022) Deze bachelorproef richt zich daarom op het ontwikkelen van een *aparte webapplicatie* (een dashboard) die integreert met GitHub-repositories van het bedrijf. Per project houdt deze applicatie de gebruikte dependencies bij, controleert via publieke registries (zoals npm,³) of er nieuwe versies beschikbaar zijn en verzamelt de relevante changelogs en release notes. Een AI-agent, die in dit onderzoek *from scratch* wordt ontworpen op basis van bestaande AI-modellen maar zonder voorafgebouwd agentframework, analyseert deze informatie, genereert begrijpelijke samenvattingen en beoordeelt in welke mate een update cruciaal is (bijvoorbeeld om veiligheidsredenen) of eerder optioneel. De AI-agent geeft het kernteam via het dashboard inzicht in vragen zoals: welke dependencies zijn verouderd, wat is er precies veranderd in de nieuwe versie, zijn er bekende beveiligingsrisico's als niet wordt geüpdatet, en lijkt de update technisch laag- of hoogrisicovol. De webapplicatie leeft dus als een losstaand platform naast GitHub, maar gebruikt GitHub uitsluitend als bron voor project- en dependency-informatie en, eventueel, als kanaal om later pull requests of issues te creëren. Op die manier kan het updateproces slimmer, sneller en inzichtelijker worden gemaakt.

1.2. Onderzoeksvraag

Om het probleem van handmatig dependencybeheer en beperkte context bij updates op te lossen, wordt de volgende hoofdonderzoeksvraag geformuleerd:

Hoe kan een AI-gedreven agent worden ingezet om dependency-updates uit bestaande tools voor automatisch dependencybeheer te analyseren en te beheren, zodat het kernteam dat verantwoordelijk is voor platform- en dependencybeheer efficiënter en met minder handmatig werk dependency-updates kan verwerken?

Hieruit vloeien de volgende deelvragen voort.

¹Dependabot is de ingebouwde GitHub-tool die kwetsbare of verouderde dependencies detecteert en automatisch update-pull-requests aanmaakt. <https://github.com/dependabot>

²Renovate is een open-source tool die automatisch update-pull-requests voor dependencies aanmaakt. <https://docs.renovatebot.com>

³npm is het standaard pakketbeheerplatform voor het JavaScript-ecosysteem. <https://www.npmjs.com>

Deelvragen voor het beter begrijpen van het probleem

1. Wat zijn de belangrijkste uitdagingen en risico's bij het huidige, overwegend manuele dependency management voor het kernteam dat verantwoordelijk is voor platform- en dependencybeheer?
2. Welke bestaande tools en oplossingen voor automatisch dependencybeheer worden vandaag gebruikt, en welke sterke punten en beperkingen hebben deze oplossingen in de context van dit bedrijf?
3. Hoe leidt ophopende technische schuld op het vlak van dependency-updates ertoe dat er steeds meer tijd en inspanning nodig zijn voor het controleren en bijwerken van dependencies binnen het bedrijf?

Deelvragen richting de oplossing

4. Hoe kunnen AI-agents en workflow automatiseringsplatformen worden ingezet om informatie uit changelogs, release notes en dependency-pull-requests automatisch te interpreteren en in begrijpelijke vorm te communiceren naar het verantwoordelijke kernteam?
5. Welke functionele en niet-functionele vereisten moet een platform voor AI-gestuurd dependencybeheer vervullen, bijvoorbeeld op het vlak van integratie met versiebeheersystemen en registries, uitbreidbaarheid, beheer van regels en policies, auditlogging en performantie?
6. In welke mate voldoen geselecteerde AI-agent- of workflowplatformen aan deze vereisten, en hoe goed kunnen zij geïntegreerd worden in een proof-of-concept voor dependency management in de context van het bedrijf?

1.3. Onderzoeksdoelstelling

Het doel van dit onderzoek is om een proof-of-concept te ontwikkelen van een *aparte webapplicatie* (dashboard) die automatisch dependency-updates detecteert, analyseert en communiceert via een geïntegreerde, from-scratch ontworpen AI-agent, en om na te gaan in welke mate geselecteerde AI-agent- of workflowplatformen deze oplossing kunnen ondersteunen en voldoen aan de vereisten van het bedrijf voor geautomatiseerd dependencybeheer. Eerder onderzoek toont aan dat AI-technieken succesvol kunnen worden ingezet binnen onderhoudstaken zoals changelog-analyse en code review, wat aantoont dat deze toepassing haalbaar en relevant is. (Behutiye et al., 2024; Joshi, 2025; Kim et al., 2024; Zhu et al., 2025)

Concreet zal de oplossing:

- per project de gebruikte dependencies bijhouden;
- automatisch nagaan of er nieuwe versies beschikbaar zijn via publieke registries (zoals npm);

- changelogs en release notes analyseren met behulp van een AI-agent die samenvat wat er gewijzigd is en welke risico's of voordelen een update inhoudt;
- de verantwoordelijke developers via een dashboard informeren over de aard en impact van de updates, inclusief een korte, begrijpelijke samenvatting en een indicatie of de update cruciaal, risicovol of veilig lijkt;
- regels ondersteunen waardoor de developers gemakkelijker kan beslissen of en wanneer een dependency geüpdatet wordt, zonder dat deze beslissingen automatisch in GitHub worden afgedwongen.

De concrete technologiekeuzes worden in de verdere uitwerking van de bachelorproef gemotiveerd op basis van de requirements en de context van het bedrijf. Het onderzoek resulteert in:

- een prototype (demo) dat de werking van het systeem aantoont binnen een of enkele projecten van het bedrijf;
- een evaluatie van de toegevoegde waarde van AI bij dependency management voor de specifieke developer;
- een aanbeveling welk type platform, en eventueel welke concrete technologie, het meest geschikt is voor verdere toepassing binnen het bedrijf.

1.4. Opzet van deze bachelorproef

De bachelorproef volgt een klassieke onderzoeksstructuur en is als volgt opgebouwd:

- **Hoofdstuk 1 – Inleiding en probleemstelling:** Dit hoofdstuk beschrijft de context, de probleemstelling, de onderzoeksvragen en de doelstellingen van de bachelorproef.
- **Hoofdstuk 2 – Literatuurstudie:** In dit hoofdstuk wordt de relevante literatuur rond dependency management, bestaande tools en oplossingen, technische schuld en AI-agentframeworks en workflow-automatisering geanalyseerd.
- **Hoofdstuk 3 – Methodologie en implementatie:** Dit hoofdstuk bespreekt de onderzoeksaanpak, het ontwerp van de systeemarchitectuur, de selectie van platformen en de implementatie van de proof-of-concept.
- **Hoofdstuk 4 – Resultaten en evaluatie:** In dit hoofdstuk worden de resultaten van de implementatie en experimenten gepresenteerd, en wordt de oplossing geëvalueerd op basis van de gedefinieerde vereisten en criteria.
- **Hoofdstuk 5 – Conclusie en toekomstig werk:** Dit hoofdstuk vat de belangrijkste bevindingen samen en formuleert aanbevelingen voor verdere uitwerking en mogelijke toekomstige verbeteringen.

2

Stand van zaken

2.0.1. Dependency management en kwetsbare dependencies

Moderne softwareprojecten steunen sterk op open-source dependencies, waardoor kwetsbaarheden in externe bibliotheken rechtstreeks een impact kunnen hebben op de veiligheid en betrouwbaarheid van toepassingen. (Pashchenko et al., [2018](#)) Pashchenko et al. tonen aan dat een groot deel van de aangetroffen kwetsbare dependencies in de praktijk relatief eenvoudig verholpen kan worden door te upgraden naar een veiligere versie, maar dat ontwikkelteams deze updates vaak niet consequent doorvoeren. (Pashchenko et al., [2018](#))

In hun proefondervindelijk onderzoek onderscheiden Pashchenko et al. tussen daadwerkelijk gedeployde kwetsbare dependencies en libraries die enkel tijdens ontwikkeling of testing gebruikt worden, en stellen zij vast dat het merendeel van de reële kwetsbare dependencies door de eigen ontwikkelaars of directe onderhouders kan worden opgelost. (Pashchenko et al., [2018](#)) Dit onderstreept het belang van systematisch dependencybeheer en duidelijke inzichten in welke dependencies effectief risico vormen in productie omgevingen. (Pashchenko et al., [2018](#))

2.0.2. Technische schuld en onderhoudslast

Het uitstellen van dependency-updates draagt bij aan de opbouw van technische schuld, waardoor onderhoud en toekomstige wijzigingen steeds meer inspanning vereisen. (Behutiye et al., [2024](#); Ruiz et al., [2024](#)) Behutiye et al. analyseren de rol van technische schuld in agile omgevingen en beschrijven hoe keuzes die op korte termijn tijdsinstaat opleveren, op langere termijn leiden tot hogere kosten en complexere wijzigingen. (Behutiye et al., [2024](#))

Ruiz et al. bespreken in hun tertiaire studie dat technische-schuldmanagement de voorbije jaren een groeiend onderzoeksdomein is geworden, waarbij organisaties worstelen met het in kaart brengen, prioriteren en terugdringen van opgebouwde

schuld.(Ruiz et al., 2024) Empirisch werk van Chalmers University of Technology bevestigt dat technische schuld niet louter een theoretisch concept is, maar in industriële context concreet voelbaar is in de vorm van verminderde productiviteit, hogere foutkans en vertragingen in softwarelevering.(Besker, 2020)

2.0.3. Dependencybots en hun beperkingen

Om de handmatige last rond dependencybeheer te verlagen, zetten veel projecten dependency management bots in, zoals Dependabot en Renovate.(Erlenhov et al., 2022) Erlenhov et al. voeren een kwantitatieve en kwalitatieve studie uit naar dependencybots in open-source systemen en stellen vast dat slechts een beperkt aantal van de geanalyseerde geautomatiseerde tools voldoet aan de criteria van volwaardige “Devbots”, die autonoom bijdragen aan de codebasis.(Erlenhov et al., 2022)

Hun resultaten tonen dat projecten vaak experimenteren met meerdere bots en regelmatig wisselen, onder meer door problemen met ruis (te veel pull requests), beperkte configureerbaarheid en moeilijkheden om de impact van voorgestelde updates goed te begrijpen.(Erlenhov et al., 2022) Hoewel dependencybots het detecteren en aanmaken van update-pull-requests automatiseren, blijft de inhoudelijke beoordeling van changelogs, compatibiliteit en risico's grotendeels bij ontwikkelaars liggen, wat aansluit bij de probleemstelling van deze bachelorproef.(Erlenhov et al., 2022; Pashchenko et al., 2018)

2.0.4. AI, agents en workflow automatisering

Recente literatuur verkent hoe AI-agents en multi-agentframeworks kunnen worden ingezet om complexe taken in softwareontwikkeling en data-intensieve workflows te coördineren.(Joshi, 2025; Kim et al., 2024; Zhu et al., 2025) Joshi biedt een overzicht van autonome systemen en collaboratieve AI-agentframeworks en benadrukt dat zulke agents vooral nuttig zijn in scenario's met veel herhalende beslissingen op basis van tekstuele en gestructureerde input, zoals logs, notificaties en rapporten.(Joshi, 2025)

Kim et al. introduceren met Bel Esprit een multi-agentframework voor het opbouwen van AI-model-pijplijnen, waarbij verschillende gespecialiseerde agents samenwerken om complexere taken modulair af te handelen.(Kim et al., 2024) Zhu et al. bestuderen in OAgents empirisch welke ontwerpkeuzes leiden tot effectieve agents, en bespreken onder meer het belang van duidelijke taakafbakening, robuuste toolintegratie en feedbackloops voor betrouwbare beslissingsondersteuning.(Zhu et al., 2025) Deze inzichten zijn relevant voor het ontwerp van een AI-agent die dependency-updates interpreteert, samenvat en beslissingsvoorstellen formuleert voor een onderhoudsteam.

Naast generieke agentframeworks beschrijft Wali et al. hoe workflow-automatisering

met n8n¹ operationele efficiëntie kan verhogen door repetitieve, gegevensgedreven processen te coördineren over verschillende systemen heen.(Wali et al., 2025) Hoewel hun casus zich richt op een financiële context, illustreert de studie dat low-code workflowtools geschikt zijn om integraties met externe APIs, databronnen en notificatiekanalen te realiseren, wat ook essentieel is voor een geautomatiseerde dependency-updatepipeline.(Wali et al., 2025)

Mastra: TypeScript-framework voor AI-agents

Naast algemene concepten en framework-onafhankelijke benaderingen voor AI-agents zijn er de laatste jaren specifieke ontwikkelframeworks ontstaan die het bouwen, testen en monitoren van agents vereenvoudigen. Een recent voorbeeld is Mastra², een open-source framework uit het JavaScript- en TypeScript-ecosysteem voor het bouwen van AI-gestuurde agents en workflows.(AI, 2026) Mastra is ontworpen rond gangbare patronen voor agentic en evaluaties, met integratie in bestaande webstacks zoals Node.js en Next.js.(AI, 2025a, 2026)

In Mastra wordt een *agent* gedefinieerd als een combinatie van een taalmodel, instructies en optioneel geheugen en tools. Tools zijn gewone functies die door de agent kunnen worden aangeroepen om extra context of data op te halen (bijvoorbeeld externe APIs of interne services), waardoor complexe taken modulair kunnen worden opgebouwd.(Nwachukwu, 2025) Daarnaast biedt het framework ondersteuning voor workflows waarin meerdere stappen, tool-calls en beslissingslogica gecombineerd worden, wat nuttig is voor scenario's zoals het sequentieel verwerken van dependency-informatie en changelogs.(The AI Builders, 2025)

Een bijkomend aandachtspunt in Mastra is observability: het framework voorziet tracing en logging van modelinteracties, agentbeslissingen, tool-calls en workflowstappen, zodat ontwikkelaars kunnen analyseren hoe een agent tot een bepaalde output is gekomen en eventuele fouten of inefficiënties kunnen opsporen.(AI, 2025b) Dit sluit aan bij de nood aan transparantie en debugbaarheid in AI-ondersteunde onderhoudsprocessen, zoals bij het automatisch interpreteren van changelogs en dependency-updates.

In het kader van deze bachelorproef wordt Mastra gebruikt om een eerste AI-integratie te realiseren: een agent die via een zelf ontwikkelde tool changelog- en dependency-informatie kan ophalen, analyseren en samenvatten. Deze tool-ondersteunde agent vormt een concrete technische invulling van de in de literatuur besproken agent-concepten en dient als basis om te onderzoeken in welke mate een dergelijke benadering effectief ondersteuning biedt bij dependencybeheer in een bedrijfscontext.

¹n8n is een open-source tool om workflows en API's visueel te automatiseren.<https://n8n.io>

²Mastra is een AI-native framework voor het bouwen en orkestreren van AI-agents en workflows.<https://mastra.ai>

2.0.5. Onderzoeksniche: AI-ondersteund dependencybeheer

Samengenomen schetst de literatuur een duidelijk beeld: kwetsbare of verouderde dependencies vormen een reëel risico, maar kunnen vaak verholpen worden door updates die vandaag onvoldoende systematisch worden uitgevoerd. (Behutiye et al., 2024; Pashchenko et al., 2018) Tegelijk tonen studies naar technische schuld dat het structureel uitstellen van onderhoud, waaronder dependency updates, leidt tot een groeiende onderhoudslast en achterstand. (Behutiye et al., 2024; Besker, 2020; Ruiz et al., 2024)

Onderzoek naar dependencybots maakt duidelijk dat huidige oplossingen weliswaar updates detecteren en pull requests genereren, maar ontwikkelaars nog steeds belasten met het interpreteren van changelogs en het beoordelen van risico's en prioriteiten. (Erlenhov et al., 2022) De recente vooruitgang in AI-agents en workflow-automatisering suggereert dat een volgende stap mogelijk is: een systeem waarin een AI-agent niet alleen updates signaleert, maar ook de inhoud van release notes en changelogs analyseert, samenvat en contextafhankelijk advies geeft aan een klein kernteam dat verantwoordelijk is voor dependencybeheer. (Joshi, 2025; Kim et al., 2024; Wali et al., 2025; Zhu et al., 2025)

Deze bachelorproef positioneert zich precies in deze niche door een AI-gestuurde laag bovenop bestaande dependencybots te onderzoeken en te prototypen, met de expliciete focus op het verlagen van de manuele beoordelingslast en het beheersbaar houden van technische schuld rond dependencies in een bedrijfscontext.

3

Methodologie

Dit onderzoek volgt de opzet van een vergelijkende studie met proof-of-concept, aangevuld met literatuuronderzoek en een requirements-analyse binnen een concrete bedrijfscontext. Het doel is om enerzijds het probleem en de oplossingsruimte rond dependencybeheer in kaart te brengen, en anderzijds een apart dashboard-prototype te bouwen en technisch te evalueren binnen een realistische ontwikkelomgeving.

3.0.1. Fase 1 - Verdiepende literatuurstudie en probleemverkenning

In de eerste fase van de bachelorproef wordt de probleemcontext verder uitgediept aan de hand van een systematische literatuurstudie rond dependency management, kwetsbare dependencies, technische schuld en bestaande dependencybots. Daarnaast wordt vakliteratuur over AI-agents en workflow-automatisering bestudeerd om na te gaan welke concepten en benaderingen relevant zijn voor een AI-gestuurd dependency-dashboard.

Tijdens deze fase worden ook één of enkele representatieve projecten van het bedrijf geselecteerd en geanalyseerd (projectstructuur, gebruikte technologieën, dependency-ecosystemen, huidige processen rond updates). De belangrijkste deliverables zijn:

- een uitgewerkte en geactualiseerde literatuurstudie;
- een scherpere probleemomschrijving toegespitst op de gekozen bedrijfscontext en projecten;
- een eerste set geïnformeerde hypothesen over de verwachte impact van een AI-gestuurd dependency-dashboard.

3.0.2. Fase 2 - Requirements-analyse en architectuurontwerp

In de tweede fase wordt eerst een gerichte requirements-analyse uitgevoerd in samenwerking met de co-promotor via interviews en workshops worden functionele

en niet-functionele eisen expliciet gemaakt, zoals:

- functionele eisen: integratie met GitHub, beheer van projecten en dependencies in het dashboard, detectie van nieuwe versies via publieke registries, analyse en samenvatting van changelogs en release notes, aanduiding van kritici- teit (security, bugfix, feature), prioritering van updates;
- niet-functionele eisen: schaalbaarheid, betrouwbaarheid, traceerbaarheid (log- ging en audittrail), gebruiksvriendelijkheid en onderhoudbaarheid.

Op basis van deze requirements wordt vervolgens een systeemarchitectuur ont- worpen voor de webapplicatie en de from-scratch ontworpen AI-agent. De archi- tectuur beschrijft onder meer de globale componenten (frontend, backend, data- bank, integraties met GitHub en registries), de verantwoordelijkheden per compo- nent, de datastromen (bijvoorbeeld van dependency-informatie naar AI-analyse en terug naar het dashboard) en de plaats van eventueel ondersteunende workflow- of agentplatformen. Deliverables van fase 2 zijn:

- een requirementsdocument met duidelijke prioritering;
- een architectuurbeschrijving (diagrammen, componentbeschrijvingen, data- modellen);
- een lijst van geselecteerde technologieën en tools, gemotiveerd vanuit de re- quirements.

3.0.3. Fase 3 - Implementatie van het proof-of-concept

In fase 3 wordt de ontworpen architectuur omgezet in een werkend proof-of-concept. De kern bestaat uit een aparte webapplicatie (dashboard) waarin per project de ge- bruikte dependencies worden bijgehouden en verrijkt met informatie uit publieke registries zoals npm. De applicatie werkt vanaf het begin met één of meerdere reële GitHub-repositories van het bedrijf, zodat de proof-of-concept onmiddellijk getest wordt in een realistische context. Dependency-informatie wordt automa- tisch uit deze repositories gehaald (bijvoorbeeld via repositoryconfiguratiebestan- den) en opgeslagen in een centrale databank.

Parallel wordt de AI-agentlaag geïmplementeerd in de backend. In deze fase wor- den drie concrete AI-agentoplossingen opgezet en geïntegreerd in hetzelfde dash- board: een agent op basis van OpenAI Agent Builder, een workflow met n8n en een agentconfiguratie met Flowise. Elk van deze varianten ontvangt changelog- en release-note-informatie, genereert beknopte samenvattingen en beoordeelt de aard en vermoedelijke impact van updates (bijvoorbeeld beveiligingsfix, bugfix, nieuwe functionaliteit, potentieel brekende wijziging). De frontend visualiseert de status van dependencies, aanbevolen acties en de door de verschillende AI-varianten gegenereerde toelichting. Deliverables van fase 3 zijn:

- een werkend prototype van het dashboard met integratie naar GitHub en één dependency-registries;
- drie geïmplementeerde AI-agentvarianten (OpenAI Agent Builder, n8n, Flowise) die changelogs en release notes verwerken en samenvatten;
- technische documentatie over de implementatie (architectuur, API's, datamodellen en integratiepunten voor de agents).

Implementatie van de Mastra-agent

Voor de eerste AI-integratie in het dashboard wordt gebruikgemaakt van Mastra, een Typescript-framework om AI-agents, tools en workflows te definiëren en te orchestreren. In de proof-of-concept wordt een `dependencyAgent` opgezet die specifiek is afgestemd op het analyseren van npm-dependency-updates. Deze agent combineert een taalmodel (bijvoorbeeld `gemini-2.5-flash` of `gpt-4o-mini`) met een eigen tool `fetchNpmInfoTool`, die informatie ophaalt uit de npm-registry en bijhorende release notes van Github.

De agent is geconfigureerd met duidelijke instructies en een vast outputformaat, zodat ontwikkelaars snel een gestructureerd overzicht krijgen van nieuwe features, breaking changes, security fixes, deprecations, migratiestappen en een advies of de update veilig is. Onderstaand codefragment toont de definitie van de agent in Mastra:

De agent maakt gebruik van een custom tool `fetchNpmInfoTool` om zowel metadata uit de npm-registry als release notes uit Github op te halen. Deze tool accepteert de packagenaam en de versies `fromVersion` en `toVersion`, zoekt de repository-URL in de npm-metadata, en probeert vervolgens de relevante GitHub-releases te vinden. Als er geen release is voor de exacte tag, wordt teruggevallen op de laatste releases, zodat de agent steeds over zo volledig mogelijke changelog-informatie beschikt. Het resultaat (beschrijving, homepage, repository en samengestelde `releaseNotes`) wordt gereïaliseerd naar een gestructureerd object waar de agent verder op kan redeneren. De implementatie van de tool is weergegeven in Listing ??.

3.0.4. Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak

De vierde fase richt zich op het onderbouwen en verfijnen van de gemaakte ontwerpkeuzes aan de hand van een gerichte vergelijking van de drie uitgewerkte AI-agentoplossingen en eventuele architecturale varianten. Op basis van de requirements uit fase 2 wordt een set *eenvoudig meetbare* evaluatiecriteria opgesteld, zoals:

- gemiddelde verwerkingstijd per update (doorlooptijd van input tot samenvatting);

- technische inspanning voor opzet en onderhoud (bijvoorbeeld aantal configuratiestappen, hoeveelheid code);
- kwaliteit van samenvattingen gemeten via een eenvoudige beoordelingsschaal door de co-promotor (bijvoorbeeld 1–5 voor duidelijkheid en relevantie);
- fout- of faalratio (bijvoorbeeld aantal mislukte runs of onvolledige resultaten per tien updates).

Aan de hand van een aantal representatieve scenario's (bijvoorbeeld updates met alleen kleine bugfixes, updates met beveiligingspatches, updates met potentieel brekende wijzigingen) worden OpenAI Agent Builder, n8n en Flowise onder dezelfde omstandigheden getest en met elkaar vergeleken. De focus ligt hierbij niet op een brede marktanalyse, maar op het versterken of bijsturen van de gekozen architectuur en het bepalen welke aanpak het meest geschikt is als basis voor het verdere gebruik van het dashboard in deze specifieke use case. Deliverables van fase 4 zijn:

- experimentele beschrijvingen van de uitgeteste varianten;
- meetresultaten volgens de gedefinieerde criteria (doorlooptijd, inspanning, beoordelingsscores, foutpercentages);
- een gemotiveerde keuze of bevestiging van de uiteindelijke architectuur en AI-configuratie.

3.0.5. Fase 5 - Evaluatie en rapportering

In de laatste fase wordt het proof-of-concept geëvalueerd met de co-promotor binnen het bedrijf, aan de hand van een combinatie van demonstraties, gericht feedbackgesprekken en, waar mogelijk, beperkte praktijktesten realistische projecten. De evaluatie focust op bruikbaarheid van het dashboard, de gemeten vermindering van manuele analysetijd (bijvoorbeeld aantal geanalyseerde updates per uur vóór en na gebruik), de door de co-promotor toegekende beoordelingsscores voor duidelijkheid van de AI-samenvattingen en de mate waarin het systeem helpt bij de prioritering van updates.

De bevindingen uit deze evaluatie worden gecombineerd met de kwantitatieve resultaten van fase 4 om de onderzoeksvragen te beantwoorden en de onderzoekshypothesen te toetsen. Deliverables van fase 5 zijn:

- een synthese van de evaluatie met de co-promotor (inclusief gemeten tijds-winst, kwaliteitsbeoordelingen en verbetervoorstellen);
- de uitgewerkte bachelorproef met een geïntegreerde beschrijving van probleem, methode, resultaten en conclusies;

- aanbevelingen voor verdere ontwikkeling en mogelijke integratie van het systeem in de bredere ontwikkelprocessen van het bedrijf.

De globale fasering en tijdsplanning van het onderzoek wordt weergegeven in Figuur A.1.



Figuur 3.1: Fasering en tijdsplanning van het onderzoek

4

Conclusie

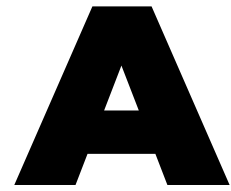
Op basis van de probleemstelling en onderzoeksdoelstelling wordt verwacht dat het proof-of-concept in staat zal zijn om dependency-updates automatisch te detecteren, te analyseren en in een compacte, begrijpelijke vorm te presenteren. De AI-agent zal changelogs en release notes samenvatten, het type wijziging (bijvoorbeeld beveiligingsfix, bugfix of nieuwe functionaliteit) aanduiden en een indicatie geven van de vermoedelijke impact op het project.

Verder wordt verwacht dat de werklast daalt doordat een deel van de beslissingen rond relatief veilige, backward compatible updates (zoals patch- of beperkte minor-releases) eenvoudiger of gedeeltelijk geautomatiseerd kan worden, terwijl potentieel risicovolle major-updates expliciet voor manuele review gemarkeerd blijven. Indien er voldoende meetdata beschikbaar komt (bijvoorbeeld logbestanden of tijdsregistraties), kan een eerste, kwantitatieve inschatting gemaakt worden van de tijdswinst in vergelijking met de huidige, hoofdzakelijk manuele aanpak.

De doelgroep, haalt hieruit meerwaarde doordat zij sneller prioriteiten kunnen bepalen, minder tijd verliezen aan het doorpluizen van changelogs en release notes en een beter overzicht krijgen van de actuele staat van dependencies en de bijhorende technische schuld. Daarnaast ontstaat een herhaalbaar proces dat later kan opgeschaald worden naar andere teams of projecten binnen de organisatie.

Verwacht wordt dat de evaluatie van de gekozen AI-agent- en/of workflowaanpak zal tonen dat geen enkele oplossing op alle criteria de beste is, maar dat duidelijke verschillen zichtbaar worden op het vlak van integratiemogelijkheden, gebruiksgemak, onderhoudbaarheid en performantie. Op basis van deze inzichten kan een onderbouwde aanbeveling worden geformuleerd voor een architectuur en toolkeuze die het best aansluit bij de context van het bedrijf, en kan geconcludeerd worden dat een AI-gestuurde laag een zinvolle stap is in het beperken van technische schuld rond dependencies. Indien de uiteindelijke resultaten afwijken van deze verwachtingen, biedt dit aanleiding om te onderzoeken welke aannames niet bleken

te kloppen en welke randvoorwaarden essentieel zijn voor succesvol AI-ondersteund dependencybeheer.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Softwareprojecten binnen bedrijven en organisaties maken gebruik van talrijke externe bibliotheken, frameworks en tools, de zogenaamde *dependencies*. Deze dependencies worden voortdurend bijgewerkt om nieuwe functionaliteiten, verbeterde prestaties en vooral beveiligingspatches aan te bieden. (Pashchenko et al., 2018) In de praktijk blijkt het voor ontwikkelteams echter moeilijk om alle updates systematisch op te volgen en grondig te evalueren, zeker wanneer een bedrijf meerdere projecten en repositories beheert.

A.1.1. Probleemstelling

Het manueel opvolgen van dependency-updates vormt een groot probleem in softwareontwikkeling. Ontwikkelaars moeten regelmatig changelogs doorzoeken, compatibiliteit controleren en inschatten welke updates veilig kunnen worden doorgevoerd, wat in de praktijk veel tijd vraagt en vaak wordt uitgesteld. (Behutiye et al., 2024; Pashchenko et al., 2018) Onderzoek toont aan dat veel softwareprojecten hun dependencies niet systematisch bijhouden, waardoor een aanzienlijk deel verouderd raakt. Pashchenko et al. (Pashchenko et al., 2018) stellen bijvoorbeeld dat een belangrijk percentage kwetsbare dependencies relatief eenvoudig opgelost kan worden door een update, maar dat dit in de praktijk niet gebeurt wegens tijdsdruk en gebrek aan overzicht. Wanneer het updateproces niet systematisch gebeurt, stapelen onderhoudstaken zich op, waardoor het steeds moeilijker en tijdrovender wordt om de software up-to-date te houden. Dit fenomeen staat bekend als

technische schuld (technical debt): de achterstand die ontstaat doordat noodzakelijke verbeteringen of updates te lang worden uitgesteld, wat leidt tot hogere onderhoudskosten en complexiteit op lange termijn.(Behutiye et al., 2024; Ruiz et al., 2024) Empirisch onderzoek bevestigt dat technische schuld binnen bedrijven reële negatieve gevolgen heeft voor productiviteit en softwarekwaliteit.(Besker, 2020) Om deze last te verlichten, zetten veel organisaties al geautomatiseerde dependencybots in, zoals Dependabot¹ of Renovate.² Deze tools maken wel automatisch pull requests aan zodra er nieuwe versies beschikbaar zijn,(Erlenhov et al., 2022) maar geven doorgaans weinig context over de inhoud en impact van de wijziging. Ontwikkelaars verliezen daardoor nog steeds tijd aan het interpreteren van changelogs, het inschatten van risico's en het beslissen of een update al dan niet moet worden doorgevoerd.(Erlenhov et al., 2022) Deze bachelorproef richt zich daarom op het ontwikkelen van een *aparte webapplicatie* (een dashboard) die integreert met GitHub-repositories van het bedrijf. Per project houdt deze applicatie de gebruikte dependencies bij, controleert via publieke registries (zoals npm,³) of er nieuwe versies beschikbaar zijn en verzamelt de relevante changelogs en release notes. Een AI-agent, die in dit onderzoek *from scratch* wordt ontworpen op basis van bestaande AI-modellen maar zonder voorafgebouwd agentframework, analyseert deze informatie, genereert begrijpelijke samenvattingen en beoordeelt in welke mate een update cruciaal is (bijvoorbeeld om veiligheidsredenen) of eerder optioneel. De AI-agent geeft het kernteam via het dashboard inzicht in vragen zoals: welke dependencies zijn verouderd, wat is er precies veranderd in de nieuwe versie, zijn er bekende beveiligingsrisico's als niet wordt geüpdatet, en lijkt de update technisch laag- of hoogrisicovol. De webapplicatie leeft dus als een losstaand platform naast GitHub, maar gebruikt GitHub uitsluitend als bron voor project- en dependency-informatie en, eventueel, als kanaal om later pull requests of issues te creëren. Op die manier kan het updateproces slimmer, sneller en inzichtelijker worden gemaakt.

A.1.2. Onderzoeksvraag

Om het probleem van handmatig dependencybeheer en beperkte context bij updates op te lossen, wordt de volgende hoofdonderzoeksvraag geformuleerd:

Hoe kan een AI-gedreven agent worden ingezet om dependency-updates uit bestaande tools voor automatisch dependencybeheer te analyseren en te beheren, zodat het kernteam dat verantwoordelijk is voor platform-

¹Dependabot is de ingebouwde GitHub-tool die kwetsbare of verouderde dependencies detecteert en automatisch update-pull-requests aanmaakt.<https://github.com/dependabot>

²Renovate is een open-source tool die automatisch update-pull-requests voor dependencies aanmaakt.<https://docs.renovatebot.com>

³npm is het standaard pakketbeheerplatform voor het JavaScript-ecosysteem.<https://www.npmjs.com>

en dependencybeheer efficiënter en met minder handmatig werk dependency-updates kan verwerken?

Hieruit vloeien de volgende deelvragen voort.

Deelvragen voor het beter begrijpen van het probleem

1. Wat zijn de belangrijkste uitdagingen en risico's bij het huidige, overwegend manuele dependency management voor het kernteam dat verantwoordelijk is voor platform- en dependencybeheer?
2. Welke bestaande tools en oplossingen voor automatisch dependencybeheer worden vandaag gebruikt, en welke sterke punten en beperkingen hebben deze oplossingen in de context van dit bedrijf?
3. Hoe leidt ophopende technische schuld op het vlak van dependency-updates ertoe dat er steeds meer tijd en inspanning nodig zijn voor het controleren en bijwerken van dependencies binnen het bedrijf?

Deelvragen richting de oplossing

4. Hoe kunnen AI-agents en workflow automatiseringsplatformen worden ingezet om informatie uit changelogs, release notes en dependency-pull-requests automatisch te interpreteren en in begrijpelijke vorm te communiceren naar het verantwoordelijke kernteam?
5. Welke functionele en niet-functionele vereisten moet een platform voor AI-gestuurd dependencybeheer vervullen, bijvoorbeeld op het vlak van integratie met versiebeheersystemen en registries, uitbreidbaarheid, beheer van regels en policies, auditlogging en performantie?
6. In welke mate voldoen geselecteerde AI-agent- of workflowplatformen aan deze vereisten, en hoe goed kunnen zij geïntegreerd worden in een proof-of-concept voor dependency management in de context van het bedrijf?

A.1.3. Onderzoeksdoelstelling

Het doel van dit onderzoek is om een proof-of-concept te ontwikkelen van een *aparte webapplicatie* (dashboard) die automatisch dependency-updates detecteert, analyseert en communiceert via een geïntegreerde, from-scratch ontworpen AI-agent, en om na te gaan in welke mate geselecteerde AI-agent- of workflowplatformen deze oplossing kunnen ondersteunen en voldoen aan de vereisten van het bedrijf voor geautomatiseerd dependencybeheer. Eerder onderzoek toont aan dat AI-technieken succesvol kunnen worden ingezet binnen onderhoudstaken zoals changelog-analyse en code review, wat aantoont dat deze toepassing haalbaar en relevant is. (Behutiye et al., 2024; Joshi, 2025; Kim et al., 2024; Zhu et al., 2025)

Concreet zal de oplossing:

- per project de gebruikte dependencies bijhouden;
- automatisch nagaan of er nieuwe versies beschikbaar zijn via publieke registers (zoals npm);
- changelogs en release notes analyseren met behulp van een AI-agent die samenvat wat er gewijzigd is en welke risico's of voordelen een update inhoudt;
- de verantwoordelijke developers via een dashboard informeren over de aard en impact van de updates, inclusief een korte, begrijpelijke samenvatting en een indicatie of de update cruciaal, risicovol of veilig lijkt;
- regels ondersteunen waardoor de developers gemakkelijker kan beslissen of en wanneer een dependency geüpdatet wordt, zonder dat deze beslissingen automatisch in GitHub worden afgedwongen.

De concrete technologiekeuzes worden in de verdere uitwerking van de bachelorproef gemotiveerd op basis van de requirements en de context van het bedrijf. Het onderzoek resulteert in:

- een prototype (demo) dat de werking van het systeem aantoont binnen een of enkele projecten van het bedrijf;
- een evaluatie van de toegevoegde waarde van AI bij dependency management voor de specifieke developer;
- een aanbeveling welk type platform, en eventueel welke concrete technologie, het meest geschikt is voor verdere toepassing binnen het bedrijf.

A.2. Literatuurstudie

A.2.1. Dependency management en kwetsbare dependencies

Moderne softwareprojecten steunen sterk op open-source dependencies, waardoor kwetsbaarheden in externe bibliotheken rechtstreeks een impact kunnen hebben op de veiligheid en betrouwbaarheid van toepassingen. (Pashchenko et al., 2018) Pashchenko et al. tonen aan dat een groot deel van de aangetroffen kwetsbare dependencies in de praktijk relatief eenvoudig verholpen kan worden door te upgraden naar een veiligere versie, maar dat ontwikkelteams deze updates vaak niet consequent doorvoeren. (Pashchenko et al., 2018)

In hun proefondervindelijk onderzoek onderscheiden Pashchenko et al. tussen daadwerkelijk gedeployde kwetsbare dependencies en libraries die enkel tijdens ontwikkeling of testing gebruikt worden, en stellen zij vast dat het merendeel van de reële kwetsbare dependencies door de eigen ontwikkelaars of directe onderhouders kan worden opgelost. (Pashchenko et al., 2018) Dit onderstreept het belang van systematisch dependencybeheer en duidelijke inzichten in welke dependencies effectief risico vormen in productie omgevingen. (Pashchenko et al., 2018)

A.2.2. Technische schuld en onderhoudslast

Het uitstellen van dependency-updates draagt bij aan de opbouw van technische schuld, waardoor onderhoud en toekomstige wijzigingen steeds meer inspanning vereisen.(Behutiye et al., 2024; Ruiz et al., 2024) Behutiye et al. analyseren de rol van technische schuld in agile omgevingen en beschrijven hoe keuzes die op korte termijn tijdswinst opleveren, op langere termijn leiden tot hogere kosten en complexere wijzigingen.(Behutiye et al., 2024)

Ruiz et al. bespreken in hun tertiaire studie dat technische-schuldmanagement de voorbije jaren een groeiend onderzoeksdomein is geworden, waarbij organisaties worstelen met het in kaart brengen, prioriteren en terugdringen van opgebouwde schuld.(Ruiz et al., 2024) Empirisch werk van Chalmers University of Technology bevestigt dat technische schuld niet louter een theoretisch concept is, maar in industriële context concreet voelbaar is in de vorm van verminderde productiviteit, hogere foutkans en vertragingen in softwarelevering.(Besker, 2020)

A.2.3. Dependencybots en hun beperkingen

Om de handmatige last rond dependencybeheer te verlagen, zetten veel projecten dependency management bots in, zoals Dependabot en Renovate.(Erlenhov et al., 2022) Erlenhov et al. voeren een kwantitatieve en kwalitatieve studie uit naar dependencybots in open-source systemen en stellen vast dat slechts een beperkt aantal van de geanalyseerde geautomatiseerde tools voldoet aan de criteria van volwaardige “Devbots”, die autonoom bijdragen aan de codebasis.(Erlenhov et al., 2022)

Hun resultaten tonen dat projecten vaak experimenteren met meerdere bots en regelmatig wisselen, onder meer door problemen met ruis (te veel pull requests), beperkte configureerbaarheid en moeilijkheden om de impact van voorgestelde updates goed te begrijpen.(Erlenhov et al., 2022) Hoewel dependencybots het detecteren en aanmaken van update-pull-requests automatiseren, blijft de inhoudelijke beoordeling van changelogs, compatibiliteit en risico's grotendeels bij ontwikkelaars liggen, wat aansluit bij de probleemstelling van deze bachelorproef.(Erlenhov et al., 2022; Pashchenko et al., 2018)

A.2.4. AI, agents en workflow automatisering

Recente literatuur verkent hoe AI-agents en multi-agentframeworks kunnen worden ingezet om complexe taken in softwareontwikkeling en data-intensieve workflows te coördineren.(Joshi, 2025; Kim et al., 2024; Zhu et al., 2025) Joshi biedt een overzicht van autonome systemen en collaboratieve AI-agentframeworks en benadrukt dat zulke agents vooral nuttig zijn in scenario's met veel herhalende beslissingen op basis van tekstuele en gestructureerde input, zoals logs, notificaties en rapporten.(Joshi, 2025)

Kim et al. introduceren met Bel Esprit een multi-agentframework voor het opbou-

wen van AI-model-pijplijnen, waarbij verschillende gespecialiseerde agents samenwerken om complexere taken modulair af te handelen.(Kim et al., 2024) Zhu et al. bestuderen in OAgents empirisch welke ontwerpkeuzes leiden tot effectieve agents, en bespreken onder meer het belang van duidelijke taakafbakening, robuuste toolintegratie en feedbackloops voor betrouwbare beslissingsondersteuning.(Zhu et al., 2025) Deze inzichten zijn relevant voor het ontwerp van een AI-agent die dependency-updates interpreteert, samenvat en beslissingsvoorstellen formuleert voor een onderhoudsteam.

Naast generieke agentframeworks beschrijft Wali et al. hoe workflow-automatisering met n8n⁴ operationele efficiëntie kan verhogen door repetitieve, gegevensgedreven processen te coördineren over verschillende systemen heen.(Wali et al., 2025) Hoewel hun casus zich richt op een financiële context, illustreert de studie dat low-code workflowtools geschikt zijn om integraties met externe APIs, databronnen en notificatiekanalen te realiseren, wat ook essentieel is voor een geautomatiseerde dependency-updatepipeline.(Wali et al., 2025)

A.2.5. Onderzoeksniche: AI-ondersteund dependencybeheer

Samengenomen schetst de literatuur een duidelijk beeld: kwetsbare of verouderde dependencies vormen een reëel risico, maar kunnen vaak verholpen worden door updates die vandaag onvoldoende systematisch worden uitgevoerd.(Behutiye et al., 2024; Pashchenko et al., 2018) Tegelijk tonen studies naar technische schuld dat het structureel uitstellen van onderhoud, waaronder dependency updates, leidt tot een groeiende onderhoudslast en achterstand.(Behutiye et al., 2024; Besker, 2020; Ruiz et al., 2024)

Onderzoek naar dependencybots maakt duidelijk dat huidige oplossingen weliswaar updates detecteren en pull requests genereren, maar ontwikkelaars nog steeds belasten met het interpreteren van changelogs en het beoordelen van risico's en prioriteiten.(Erlenhov et al., 2022) De recente vooruitgang in AI-agents en workflow-automatisering suggereert dat een volgende stap mogelijk is: een systeem waarin een AI-agent niet alleen updates signaleert, maar ook de inhoud van release notes en changelogs analyseert, samenvat en contextafhankelijk advies geeft aan een klein kernteam dat verantwoordelijk is voor dependencybeheer.(Joshi, 2025; Kim et al., 2024; Wali et al., 2025; Zhu et al., 2025)

Deze bachelorproef positioneert zich precies in deze niche door een AI-gestuurde laag bovenop bestaande dependencybots te onderzoeken en te prototypen, met de expliciete focus op het verlagen van de manuele beoordelingslast en het beheersbaar houden van technische schuld rond dependencies in een bedrijfscontext.

⁴n8n is een open-source tool om workflows en API's visueel te automatiseren.<https://n8n.io>

A.3. Methodologie

Dit onderzoek volgt de opzet van een vergelijkende studie met proof-of-concept, aangevuld met literatuuronderzoek en een requirements-analyse binnen een concrete bedrijfscontext. Het doel is om enerzijds het probleem en de oplossingsruimte rond dependencybeheer in kaart te brengen, en anderzijds een apart dashboard-prototype te bouwen en technisch te evalueren binnen een realistische ontwikkelomgeving.

A.3.1. Fase 1 - Verdiepende literatuurstudie en probleemverkenning

In de eerste fase van de bachelorproef wordt de probleemcontext verder uitgediept aan de hand van een systematische literatuurstudie rond dependency management, kwetsbare dependencies, technische schuld en bestaande dependencybots. Daarnaast wordt vakliteratuur over AI-agents en workflow-automatisering bestudeerd om na te gaan welke concepten en benaderingen relevant zijn voor een AI-gestuurd dependency-dashboard.

Tijdens deze fase worden ook één of enkele representatieve projecten van het bedrijf geselecteerd en geanalyseerd (projectstructuur, gebruikte technologieën, dependency-ecosystemen, huidige processen rond updates). De belangrijkste deliverables zijn:

- een uitgewerkte en geactualiseerde literatuurstudie;
- een scherpere probleemomschrijving toegespitst op de gekozen bedrijfscontext en projecten;
- een eerste set geïnformeerde hypothesen over de verwachte impact van een AI-gestuurd dependency-dashboard.

A.3.2. Fase 2 - Requirements-analyse en architectuurontwerp

In de tweede fase wordt eerst een gerichte requirements-analyse uitgevoerd in samenwerking met de co-promotor via interviews en workshops worden functionele en niet-functionele eisen expliciet gemaakt, zoals:

- functionele eisen: integratie met GitHub, beheer van projecten en dependencies in het dashboard, detectie van nieuwe versies via publieke registries, analyse en samenvatting van changelogs en release notes, aanduiding van kriticiiteit (security, bugfix, feature), prioritering van updates;
- niet-functionele eisen: schaalbaarheid, betrouwbaarheid, traceerbaarheid (logging en audittrail), gebruiksvriendelijkheid en onderhoudbaarheid.

Op basis van deze requirements wordt vervolgens een systeemarchitectuur ontworpen voor de webapplicatie en de from-scratch ontworpen AI-agent. De architectuur beschrijft onder meer de globale componenten (frontend, backend, databank, integraties met GitHub en registries), de verantwoordelijkheden per component, de datastromen (bijvoorbeeld van dependency-informatie naar AI-analyse en

terug naar het dashboard) en de plaats van eventueel ondersteunende workflow- of agentplatformen. Deliverables van fase 2 zijn:

- een requirementsdocument met duidelijke prioritering;
- een architectuurbeschrijving (diagrammen, componentbeschrijvingen, data-modellen);
- een lijst van geselecteerde technologieën en tools, gemotiveerd vanuit de requirements.

A.3.3. Fase 3 - Implementatie van het proof-of-concept

In fase 3 wordt de ontworpen architectuur omgezet in een werkend proof-of-concept. De kern bestaat uit een aparte webapplicatie (dashboard) waarin per project de gebruikte dependencies worden bijgehouden en verrijkt met informatie uit publieke registries zoals npm. De applicatie werkt vanaf het begin met één of meerdere reële GitHub-repositories van het bedrijf, zodat de proof-of-concept onmiddellijk getest wordt in een realistische context. Dependency-informatie wordt automatisch uit deze repositories gehaald (bijvoorbeeld via repositoryconfiguratiebestanden) en opgeslagen in een centrale databank.

Parallel wordt de AI-agentlaag geïmplementeerd in de backend. In deze fase worden drie concrete AI-agentoplossingen opgezet en geïntegreerd in hetzelfde dashboard: een agent op basis van OpenAI Agent Builder, een workflow met n8n en een agentconfiguratie met Flowise. Elk van deze varianten ontvangt changelogs en release-note-informatie, genereert beknopte samenvattingen en beoordeelt de aard en vermoedelijke impact van updates (bijvoorbeeld beveiligingsfix, bugfix, nieuwe functionaliteit, potentieel brekende wijziging). De frontend visualiseert de status van dependencies, aanbevolen acties en de door de verschillende AI-varianten gegenereerde toelichting. Deliverables van fase 3 zijn:

- een werkend prototype van het dashboard met integratie naar GitHub en één dependency-registries;
- drie geïmplementeerde AI-agentvarianten (OpenAI Agent Builder, n8n, Flowise) die changelogs en release notes verwerken en samenvatten;
- technische documentatie over de implementatie (architectuur, API's, datamodellen en integratiepunten voor de agents).

A.3.4. Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak

De vierde fase richt zich op het onderbouwen en verfijnen van de gemaakte ontwerpkeuzes aan de hand van een gerichte vergelijking van de drie uitgewerkte AI-agentoplossingen en eventuele architecturale varianten. Op basis van de requi-

rements uit fase 2 wordt een set *eenvoudig meetbare* evaluatiecriteria opgesteld, zoals:

- gemiddelde verwerkingstijd per update (doorlooptijd van input tot samenvatting);
- technische inspanning voor opzet en onderhoud (bijvoorbeeld aantal configuratiestappen, hoeveelheid code);
- kwaliteit van samenvattingen gemeten via een eenvoudige beoordelingsschaal door de co-promotor (bijvoorbeeld 1–5 voor duidelijkheid en relevantie);
- fout- of faalratio (bijvoorbeeld aantal mislukte runs of onvolledige resultaten per tien updates).

Aan de hand van een aantal representatieve scenario's (bijvoorbeeld updates met alleen kleine bugfixes, updates met beveiligingspatches, updates met potentieel brekende wijzigingen) worden OpenAI Agent Builder, n8n en Flowise onder dezelfde omstandigheden getest en met elkaar vergeleken. De focus ligt hierbij niet op een brede marktanalyse, maar op het versterken of bijsturen van de gekozen architectuur en het bepalen welke aanpak het meest geschikt is als basis voor het verdere gebruik van het dashboard in deze specifieke use case. Deliverables van fase 4 zijn:

- experimentele beschrijvingen van de uitgeteste varianten;
- meetresultaten volgens de gedefinieerde criteria (doorlooptijd, inspanning, beoordelingsscores, foutpercentages);
- een gemotiveerde keuze of bevestiging van de uiteindelijke architectuur en AI-configuratie.

A.3.5. Fase 5 - Evaluatie en rapportering

In de laatste fase wordt het proof-of-concept geëvalueerd met de co-promotor binnen het bedrijf, aan de hand van een combinatie van demonstraties, gericht feedbackgesprekken en, waar mogelijk, beperkte praktijktesten realistische projecten. De evaluatie focust op bruikbaarheid van het dashboard, de gemeten vermindering van manuele analysetijd (bijvoorbeeld aantal geanalyseerde updates per uur vóór en na gebruik), de door de co-promotor toegekende beoordelingsscores voor duidelijkheid van de AI-samenvattingen en de mate waarin het systeem helpt bij de prioritering van updates.

De bevindingen uit deze evaluatie worden gecombineerd met de kwantitatieve resultaten van fase 4 om de onderzoeksvragen te beantwoorden en de onderzoekshypothesen te toetsen. Deliverables van fase 5 zijn:

- een synthese van de evaluatie met de co-promotor (inclusief gemeten tijdswinst, kwaliteitsbeoordelingen en verbetervoorstellen);
- de uitgewerkte bachelorproef met een geïntegreerde beschrijving van probleem, methode, resultaten en conclusies;
- aanbevelingen voor verdere ontwikkeling en mogelijke integratie van het systeem in de bredere ontwikkelprocessen van het bedrijf.

De globale fasering en tijdsplanning van het onderzoek wordt weergegeven in Figuur A.1.

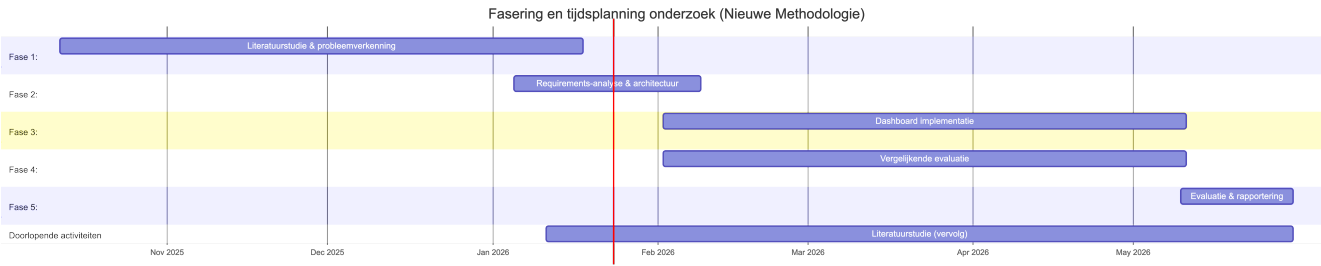
A.4. Verwacht resultaat en conclusie

Op basis van de probleemstelling en onderzoeksdoelstelling wordt verwacht dat het proof-of-concept in staat zal zijn om dependency-updates automatisch te detecteren, te analyseren en in een compacte, begrijpelijke vorm te presenteren. De AI-agent zal changelogs en release notes samenvatten, het type wijziging (bijvoorbeeld beveiligingsfix, bugfix of nieuwe functionaliteit) aanduiden en een indicatie geven van de vermoedelijke impact op het project.

Verder wordt verwacht dat de werklast daalt doordat een deel van de beslissingen rond relatief veilige, backward compatible updates (zoals patch- of beperkte minor-releases) eenvoudiger of gedeeltelijk geautomatiseerd kan worden, terwijl potentieel risicovolle major-updates expliciet voor manuele review gemarkeerd blijven. Indien er voldoende meetdata beschikbaar komt (bijvoorbeeld logbestanden of tijdsregistraties), kan een eerste, kwantitatieve inschatting gemaakt worden van de tijdswinst in vergelijking met de huidige, hoofdzakelijk manuele aanpak.

De doelgroep, haalt hieruit meerwaarde doordat zij sneller prioriteiten kunnen bepalen, minder tijd verliezen aan het doorpluizen van changelogs en release notes en een beter overzicht krijgen van de actuele staat van dependencies en de bijhorende technische schuld. Daarnaast ontstaat een herhaalbaar proces dat later kan opgeschaald worden naar andere teams of projecten binnen de organisatie.

Verwacht wordt dat de evaluatie van de gekozen AI-agent- en/of workflowaanpak zal tonen dat geen enkele oplossing op alle criteria de beste is, maar dat duidelijke verschillen zichtbaar worden op het vlak van integratiemogelijkheden, gebruiksgemak, onderhoudbaarheid en performantie. Op basis van deze inzichten kan een onderbouwde aanbeveling worden geformuleerd voor een architectuur en toolkeuze die het best aansluit bij de context van het bedrijf, en kan geconcludeerd worden dat een AI-gestuurde laag een zinvolle stap is in het beperken van technische schuld rond dependencies. Indien de uiteindelijke resultaten afwijken van deze verwachtingen, biedt dit aanleiding om te onderzoeken welke aannames niet bleken te kloppen en welke randvoorwaarden essentieel zijn voor succesvol AI-ondersteund dependencybeheer.



Figuur A.1: Fasering en tijdsplanning van het onderzoek

Bibliografie

- AI, M. (2025a). *Mastra: The Javascript framework for building AI agents* [Beschrijving van Mastra als JavaScript-framework voor AI-agents met workflows, memory en evals]. Geraadpleegd op 2 maart 2026, van <https://www.ycombinator.com/companies/mastra>
- AI, M. (2025b). *Observability Overview* [Documentatie over tracing en observability voor Mastra-agents en workflows]. Geraadpleegd op 2 maart 2026, van <https://mastra.ai/docs/observability/overview>
- AI, M. (2026). *Mastra: The TypeScript AI Agent Framework* [Open-source framework voor het bouwen van AI-agents en workflows in TypeScript]. Geraadpleegd op 2 maart 2026, van <https://github.com/mastra-ai/mastra>
- Behutiye, W. N., Rodriguez, P., Oivo, M., & Tosun, A. (2024). Analyzing the Concept of Technical Debt in the Context of Agile Software Development: A Systematic Literature Review. *arXiv*. <https://arxiv.org/abs/2401.14882>
- Besker, T. (2020). *Technical Debt: An Empirical Investigation of Its Harmfulness and Management Strategies in Industry* [PhD thesis]. Chalmers University of Technology. <https://research.chalmers.se/en/publication/518318>
- Erlenhov, L., De Oliveira Neto, F. G., & Leitner, P. (2022). Dependency Management Bots in Open-Source Systems—Prevalence and Adoption. *PeerJ Computer Science*, 8, e849. <https://doi.org/10.7717/peerj-cs.849>
- Joshi, S. (2025). Review of Autonomous Systems and Collaborative AI Agent Frameworks. *International Journal of Science and Research Archive*, 14(02), 961–972. <https://doi.org/10.30574/ijrsra.2025.14.2.0439>
- Kim, Y., Abdelaziz, A. E., Castro Ferreira, T., Al-Badrashiny, M., & Sawaf, H. (2024). Bel Esprit: Multi-Agent Framework for Building AI Model Pipelines. *arXiv*. <https://doi.org/10.48550/arXiv.2412.14684>
- Nwachukwu, U. (2025). *Creating AI Agents with Mastra and TypeScript* [Artikel dat uitlegt hoe Mastra-agents en tools worden gedefinieerd in TypeScript]. Geraadpleegd op 2 maart 2026, van <https://dev.to/ucodes/creating-ai-agents-with-mastra-and-typescript-4d6o>
- Pashchenko, I., Plate, H., Ponta, S. E., Sabetta, A., & Massacci, F. (2018). Vulnerable Open Source Dependencies: Counting Those That Matter. *Proceedings of the 12th International Symposium on Empirical Software Engineering and Measurement (ESEM)*.

- Ruiz, J. A., Aguilar, R. A., Garcilazo, J. F., & Aguilera, A. A. (2024). A Tertiary Study on Technical Debt Management over the last lustrum. *International Journal of Combinatorial Optimization Problems and Informatics*, 15(5), 181–192. <https://doi.org/10.61467/2007.1558.2024.v15i5.577>
- The AI Builders. (2025). *Mastra: Build Powerful AI Agents with Typescript* [Tutorial met voorbeeldagents, tools en workflows in Mastra]. Geraadpleegd op 2 maart 2026, van https://tutorial.theaibuilders.dev/tutorials/Frameworks/mastra_tutorial
- Wali, M., Nasir, N., & Iqbal, T. (2025). Implementing Workflow Automation with n8n to Enhance Operational Efficiency and Performance in the Sharia Cooperative of Bank Indonesia, Aceh Province. *Journal Digital Technology Trend*, 4(1), 36–47. <https://doi.org/10.56347/jdtt.v4i1.341>
- Zhu, H., Qin, T., Zhu, K., Huang, H., Guan, Y., Xia, J., Yao, Y., Li, H., Wang, N., Liu, P., Peng, T., Gui, X., Li, X., Liu, Y., Jiang, Y. E., Wang, J., Zhang, C., Tang, X., Zhang, G., ... Zhou, W. (2025). OAgents: An Empirical Study of Building Effective Agents. <https://arxiv.org/abs/2506.15741>